

Nama: Muhammad Akmal Dwiansyah Putra

Kelas / Absen: TI_1G / 20

NIM: 254107020110

Praktikum Jobsheet 14

Praktikum 1:

Mahasiswa20.java:

```
package Jobsheet14;
```

```
public class Mahasiswa20 {
```

```
    String nim;
```

```
    String nama;
```

```
    String kelas;
```

```
    double ipk;
```

```
    Mahasiswa20(){
```

```
}
```

```
    Mahasiswa20(String nim, String nama, String kelas, double ipk){
```

```
        this.nim = nim;
```

```
        this.nama = nama;
```

```
        this.kelas = kelas;
```

```
        this.ipk = ipk;
```

```
}
```

```
    void tampilInformasi(){
```

```
        System.out.println("NIM : " + nim);
```

```
        System.out.println("Nama : " + nama);
```

```
        System.out.println("Kelas: " + kelas);
```

```
        System.out.println("IPK : " + ipk);
        System.out.println("=====");
    }
}
```

Node20.java:

```
package Jobsheet14;

public class Node20 {
    Mahasiswa20 mahasiswa;
    Node20 left;
    Node20 right;

    Node20(){

    }

    Node20(Mahasiswa20 mahasiswa){
        this.mahasiswa = mahasiswa;
        left = right = null;
    }
}
```

BinaryTree20.java:

```
package Jobsheet14;

public class BinaryTree20 {
    Node20 root;

    BinaryTree20(){
```

```
    root = null;
}
```

```
boolean isEmpty(){
    return root == null;
}
```

```
void add(Mahasiswa20 mahasiswa){
    Node20 newNode = new Node20(mahasiswa);
    if (isEmpty()) {
        root = newNode;
    }else{
        Node20 current = root;
        Node20 parent = null;
        while (true) {
            parent = current;
            if (mahasiswa.ipk < current.mahasiswa.ipk) {
                current = current.left;
                if (current == null) {
                    parent.left = newNode;
                    return;
                }
            }else{
                current = current.right;
                if (current == null) {
                    parent.right = newNode;
                    return;
                }
            }
        }
    }
}
```

```
}  
}
```

```
boolean found(double ipk){  
    boolean result = false;  
    Node20 current = root;  
    while (current != null) {  
        if (current.mahasiswa.ipk == ipk) {  
            result = true;  
            break;  
        }else if (current.mahasiswa.ipk < ipk) {  
            current = current.right;  
        }else{  
            current = current.left;  
        }  
    }  
    return result;  
}
```

```
void traversePreOrder(Node20 node){  
    if (node != null) {  
        node.mahasiswa.tampilInformasi();  
        traversePreOrder(node.left);  
        traversePreOrder(node.right);  
    }  
}
```

```
void traverseInOrder(Node20 node){  
    if (node != null) {  
        traverseInOrder(node.left);
```

```

        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

```

```

void traversePostOrder(Node20 node){
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

```

```

Node20 getSuccessor(Node20 del){
    Node20 successor = del.right;
    Node20 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

```

```

void delete(double ipk){
    if (isEmpty()) {
        System.out.println("Binary Tree masih kosong");
    }
}

```

```
    return;  
}
```

```
Node20 parent = root;  
Node20 current = root;  
boolean isLeftChild = false;  
while (current != null) {  
    if (current.mahasiswa.ipk == ipk) {  
        break;  
    } else if (current.mahasiswa.ipk < ipk) {  
        parent = current;  
        current = current.left;  
        isLeftChild = true;  
    } else if (current.mahasiswa.ipk > ipk) {  
        parent = current;  
        current = current.right;  
        isLeftChild = false;  
    }  
}
```

```
if (current == null) {  
    System.out.println("Data tidak ditemukan");  
    return;  
} else {  
    if (current.left == null && current.right == null) {  
        if (current == root) {  
            root = null;  
        } else {  
            if (isLeftChild) {  
                parent.left = null;  
            }  
        }  
    }  
}
```

```

        }else{
            parent.right = null;
        }
    }
}
}else if (current.left == null) {
    if (current == root) {
        root = current.right;
    }else{
        if (isLeftChild) {
            parent.left = current.right;
        }else{
            parent.right = current.right;
        }
    }
}
}else if (current.right == null) {
    if (current == root) {
        root = current.left;
    }else{
        if (isLeftChild) {
            parent.left = current.left;
        }else{
            parent.right = current.left;
        }
    }
}
}else{
    Node20 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    }
}

```

```

    }else{
        if (isLeftChild) {
            parent.left = successor;
        }else{
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}

}
}

```

BinaryTreeMain20.java:

```

package Jobsheet14;

public class BinaryTreeMain20 {
    public static void main(String[] args) {
        BinaryTree20 bst = new BinaryTree20();

        bst.add(new Mahasiswa20("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa20("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa20("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa20("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua Mahasiswa (In Order Traverse): ");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data Mahasiswa");
    }
}

```



```
System.out.print("Cari mahasiswa dengan IPK: 3.54 -> ");  
String hasilCari = bst.found(3.54)?"Ditemukan":"Tidak ditemukan";  
System.out.println(hasilCari);
```

```
System.out.print("Cari mahasiswa dengan IPK: 3.22 -> ");  
hasilCari = bst.found(3.22)?"Ditemukan":"Tidak ditemukan";  
System.out.println(hasilCari);
```

```
bst.add(new Mahasiswa20("244160131", "Devi", "A", 3.72));  
bst.add(new Mahasiswa20("244160205", "Ehsan", "D", 3.37));  
bst.add(new Mahasiswa20("244160170", "Fizi", "B", 3.46));  
System.out.println("\nDaftar semua Mahasiswa");  
System.out.println("\nInOrder Traversal");  
bst.traverseInOrder(bst.root);  
System.out.println("\nPreOrder Traversal");  
bst.traversePreOrder(bst.root);  
System.out.println("\nPostOrder Traversal");  
bst.traversePostOrder(bst.root);
```

```
System.out.println("\nPenghapusan data");  
bst.delete(3.57);  
System.out.println("\nDaftar semua Mahasiswa");  
bst.traverseInOrder(bst.root);
```

```
}  
  
}
```

HASIL:

Daftar semua Mahasiswa (In Order Traverse):

NIM : 244160185

Nama : Candra

Kelas: C

IPK : 3.21

=====

NIM : 244160220

Nama : Dewi

Kelas: B

IPK : 3.54

=====

NIM : 244160121

Nama : Ali

Kelas: A

IPK : 3.57

=====

NIM : 244160221

Nama : Badar

Kelas: B

IPK : 3.85

=====

Pencarian data Mahasiswa

Cari mahasiswa dengan IPK: 3.54 ->Ditemukan

Cari mahasiswa dengan IPK: 3.22 -> Tidak ditemukan

Daftar semua Mahasiswa

InOrder Traversal

NIM : 244160185

Nama : Candra

Kelas: C

IPK : 3.21

=====

NIM : 244160205

Nama : Ehsan

Kelas: D

IPK : 3.37

=====

NIM : 244160170

Nama : Fizi

Kelas: B

IPK : 3.46

=====

NIM : 244160220

Nama : Dewi

Kelas: B

IPK : 3.54

=====

NIM : 244160121

Nama : Ali

Kelas: A

IPK : 3.57

=====

NIM : 244160131

Nama : Devi

Kelas: A

IPK : 3.72

=====

NIM : 244160221

Nama : Badar

Kelas: B

IPK : 3.85

=====

PreOrder Traversal

NIM : 244160121

Nama : Ali

Kelas: A

IPK : 3.57

=====

NIM : 244160185

Nama : Candra

Kelas: C

IPK : 3.21

=====

NIM : 244160220

Nama : Dewi

Kelas: B

IPK : 3.54

=====

NIM : 244160205

Nama : Ehsan

Kelas: D

IPK : 3.37

=====

NIM : 244160170

Nama : Fizi

Kelas: B

IPK : 3.46

=====

NIM : 244160221

Nama : Badar

Kelas: B

IPK : 3.85

=====

NIM : 244160131

Nama : Devi

Kelas: A

IPK : 3.72

=====

PostOrder Traversal

NIM : 244160170

Nama : Fizi

Kelas: B

IPK : 3.46

=====

NIM : 244160205

Nama : Ehsan

Kelas: D

IPK : 3.37

=====

NIM : 244160220

Nama : Dewi

Kelas: B

IPK : 3.54

=====

NIM : 244160185

Nama : Candra

Kelas: C

IPK : 3.21

=====

NIM : 244160131

Nama : Devi

Kelas: A

IPK : 3.72

=====

NIM : 244160221

Nama : Badar

Kelas: B

IPK : 3.85

=====

NIM : 244160121

Nama : Ali

Kelas: A

IPK : 3.57

=====

Penghapusan data

Jika 2 anak, current =

NIM : 244160131

Nama : Devi

Kelas: A

IPK : 3.72

=====

Daftar semua Mahasiswa

NIM : 244160185

Nama : Candra

Kelas: C

IPK : 3.21

=====

NIM : 244160205

Nama : Ehsan

Kelas: D

IPK : 3.37

=====

NIM : 244160170

Nama : Fizi

Kelas: B

IPK : 3.46

=====

NIM : 244160220

Nama : Dewi

Kelas: B

IPK : 3.54

=====

NIM : 244160131

Nama : Devi

Kelas: A

IPK : 3.72

=====

NIM : 244160221

Nama : Badar

Kelas: B

IPK : 3.85

=====

14.2.2 Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Jawaban: Binary search tree lebih efektif dari pada binary tree biasa karena memiliki pemilihan untuk menentukan apakah hal yang dicari lebih besar atau lebih kecil, jadi tidak perlu mencari semua data tetapi hanya data yang sama besarnya saja.

2. Untuk apakah di class Node, kegunaan dari atribut left dan right?

Jawaban: Kegunaan left dan right pada class node adalah sebagai pemisah, untuk hal ini adalah besar dari nilai IPK, kiri merupakan IPK kurang dari root sedangkan kanan merupakan IPK lebih dari root.

3. a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?
b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?

Jawaban: A. Kegunaan root dalam class BinaryTree adalah sebagai awal data mulai, ini digunakan Ketika melakukan hal seperti traversal

B. Ketika BinaryTree pertama kali dibuat, nilai dari root merupakan null

4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Jawaban: Ketika data masih kosong dan ditambahkan node baru, maka di lakukan pengecekan terlebih dahulu menggunakan isEmpty() dimana apakah root == null, jika iya maka root tersebut menjadi node baru.

5. Perhatikan method add(), di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

```
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

Jawaban: Potongan program tersebut digunakan untuk mengecek apakah nilai ipk dari mahasiswa yang di input lebih besar atau lebih kecil dari nilai ipk mahasiswa sebelumnya, ini dilakukan untuk memastikan letak data berada di kiri atau kanan.

6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

Jawaban: Pada Langkah pertama program melakukan pengecekan apakah data masih kosong atau tidak, setelah itu melakukan looping untuk menentukan letak dari semua data dan data yang ingin dihapus, menggunakan pengecekan Kembali untuk menentukan apakah data terdapat di kiri atau di kanan lalu dihapus. Kegunaan getSuccessor() adalah untuk menentukan keturunan dari data yang ingin dihapus, dengan ini keturunan data yang dihapus tersebut itu menjadi orang tua nya.

Praktikum 2:

BinaryTreeArray20.java:

```
package Jobsheet14;
```

```
public class BinaryTreeArray20 {  
    Mahasiswa20[] dataMahasiswa;  
    int idxLast;  
  
    BinaryTreeArray20(){  
        dataMahasiswa = new Mahasiswa20[10];  
    }  
  
    void populateData(Mahasiswa20[] data, int idxLast){  
        dataMahasiswa = data;  
        this.idxLast = idxLast;  
    }  
  
    void traverseInOrder(int idxStart){  
        if (idxStart <= idxLast) {  
            if (dataMahasiswa[idxStart] != null) {  
                traverseInOrder(2*idxStart+1);  
                dataMahasiswa[idxStart].tampilInformasi();  
                traverseInOrder(2*idxStart+2);  
            }  
        }  
    }  
}
```

```
}  
}
```

BinaryTreeArrayMain20.java:

```
package Jobsheet14;
```

```
public class BinaryTreeArrayMain20 {  
    public static void main(String[] args) {  
        BinaryTreeArray20 bta = new BinaryTreeArray20();  
        Mahasiswa20 mhs1 = new Mahasiswa20("244160121", "Ali", "A", 3.57);  
        Mahasiswa20 mhs2 = new Mahasiswa20("244160221", "Candra", "B", 3.41);  
        Mahasiswa20 mhs3 = new Mahasiswa20("244160185", "Badar", "C", 3.75);  
        Mahasiswa20 mhs4 = new Mahasiswa20("244160220", "Dewi", "B", 3.35);  
  
        Mahasiswa20 mhs5 = new Mahasiswa20("244160131", "Devi", "A", 3.48);  
        Mahasiswa20 mhs6 = new Mahasiswa20("244160205", "Ehsan", "D", 3.61);  
        Mahasiswa20 mhs7 = new Mahasiswa20("244160170", "Fizi", "B", 3.86);  
  
        Mahasiswa20[] dataMahasiswa =  
        {mhs1,mhs2,mhs3,mhs4,mhs5,mhs6,mhs7,null,null,null};  
        int idxLast = 6;  
        bta.populateData(dataMahasiswa, idxLast);  
        System.out.println("\nInOrder Traversal Mahasiswa: ");  
        bta.traverseInOrder(0);  
    }  
}
```

HASIL:

InOrder Traversal Mahasiswa

NIM : 244160220

Nama : Dewi

Kelas: B

IPK : 3.35

=====

NIM : 244160221

Nama : Candra

Kelas: B

IPK : 3.41

=====

NIM : 244160131

Nama : Devi

Kelas: A

IPK : 3.48

=====

NIM : 244160121

Nama : Ali

Kelas: A

IPK : 3.57

=====

NIM : 244160205

Nama : Ehsan

Kelas: D

IPK : 3.61

=====

NIM : 244160185

Nama : Badar

Kelas: C

IPK : 3.75

=====

NIM : 244160170

Nama : Fizi

Kelas: B

IPK : 3.86

=====

14.3.2 Pertanyaan Percobaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?

Jawaban: Atribut data digunakan sebagai template dari data yang ingin dibuat sedangkan idxLast digunakan sebagai berapa banyak data yang telah di input atau tidak null

2. Apakah kegunaan dari method populateData()?

Jawaban: Kegunaan dari populateData() adalah untuk mengisi data di BinaryTreeArray

3. Apakah kegunaan dari method traverseInOrder()?

Jawaban: Kegunaan dari traverseInOrder adalah untuk menampilkan semua data yang ada, juga mengatur order penampilan semua data tersebut.

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?

Jawaban: Posisi left child terdapat pada index 5 sedangkan posisi right child terdapat pada index ke 6

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

Jawaban: Kegunaan statement tersebut adalah menunjukkan berapa banyak data yang telah diisi, ini digunakan dalam tahap traverseInOrder.

6. Mengapa indeks $2*idxStart+1$ dan $2*idxStart+2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawaban: dalam BinaryTree, index $2*idx+1$ adalah untuk menunjukkan bahwa data diletakkan pada posisi kiri, sedangkan index $2*idx+2$ meletakkan data pada posisi kanan.